

Phase 2 Implementation Report

Smart Lead Qualification — Vyapar AI Employee Platform

Status: **IMPLEMENTED** **PLAN v2 + ADJUSTMENTS**

Date: June 2, 2026

Goal: Sales employee with lead intelligence — not a FAQ bot

Final qualified rule (approved adjustment):
qualified requires **(phone OR WhatsApp contact) AND (location OR speed)**.
Score alone cannot create qualified — max stage without contact is interested.

1. What Was Built

Capability	Module
Buying intent detection (Nepali + English)	lead_extractor.py
Lead signals: budget, location, speed, urgency, coverage	lead_extractor.py
contact_method: phone / telegram / whatsapp	lead_extractor.py
lead_score 0–100 + stages	lead_extractor.py + lead_manager.py
Lead persistence (leads table)	memory_db.py + lead_manager.py
Sales mode — no full package list dump	products.py + ai_employee_engine.py
Sales conversation memory on lead	lead_manager.py
Admin alert placeholder	admin_notifier.py
Lead export CLI	admin_export_leads.py

2. Architecture After Phase 2

```
Telegram → main.py → ai_employee_engine.py
├─ memory_extractor (unchanged)
├─ lead_extractor (NEW)
│   └─ → compute score + derive stage
├─ lead_manager (NEW) → leads table
├─ products.find_best_product_match (NEW)
├─ sales_mode? → single alternative, no catalog dump
├─ prompts: lead_block + sales_memory_block
├─ openai_engine (unchanged)
└─ admin_notifier placeholder (NEW)
```

Preserved: company_manager, COMPANY_ID, memory, Telegram webhook

3. Pipeline (16 steps)

1. sanitize + save user turn
2. extract_memory_facts
3. extract_lead_bundle (signals, score, stage)
4. upsert_lead if should_process_lead
5. direct memory answer (unchanged)
6. company config check
7. find_best_product_match
8. resolve sales_mode
9. compose prompt (sales/coverage hints)
10. OpenAI reply

- 11. update_sales_memory
- 12. maybe_notify_admin

4. Lead Stages & Scoring

Score factors (0–100, additive)

Factor	Points
Buying intent	+25
Speed mentioned	+15
Location mentioned	+15
Budget mentioned	+10
Urgency high / medium	+10 / +5
Phone / WhatsApp	+20 / +18
Name, telegram contact, coverage flag	+5 each

Stage rules (hard gates)

Stage	Requirements
new	Minimal — buying only
interested	Buying + signals OR score≥40 — NO qualified without contact
qualified	(phone OR whatsapp) AND (location OR speed)
hot	qualified + high urgency OR medium urgency + score≥75

Stage never downgrades on re-upsert.

5. Example Flow (Consistent)

```
Message 1: "Baluwatar ma 500 mbps internet jodnu cha"
  buying_intent = true
  coverage_check_needed = true
  location = Baluwatar, speed = 500 Mbps
  contact = telegram only (no phone)
  score ≈ 55
  stage = interested ← NOT qualified (no phone/WhatsApp)

Message 2: "Mero phone 9801234567 ho"
  phone added
  stage = qualified ← phone + location + speed

Message 3: "Chhito chha aile install garna cha"
  urgency = high
  stage = hot
```

6. Sales Mode — Unavailable Package

When buying intent or no exact product match:

- Do NOT inject full package catalog
- Inject single suggested alternative via `format_alternative_product()`
- Acknowledge customer need + coverage pending if ISP
- Ask for phone or WhatsApp if missing
- Update `last_discussed_product`, `last_sales_question`, `last_sales_reply` on lead

7. Files Created (5)

File	Purpose
lead_extractor.py	Intent, signals, score, stage derivation
lead_manager.py	Upsert, sales memory, export, prompt blocks
admin_notifier.py	ADMIN_ALERT_ENABLED log placeholder
admin_export_leads.py	CLI export to exports/
tests/test_lead_phase2.py	7 unit tests for checklist

8. Files Modified (7)

File	Change
memory_db.py	Lead model (25 columns)
products.py	<code>find_best_product_match</code> , <code>include_full_catalog</code> flag
prompts.py	SALES_EMPLOYEE_HINT, lead/sales blocks
intent_engine.py	<code>buying_intent</code> , <code>coverage_inquiry</code> intents
ai_employee_engine.py	Full lead pipeline integration
env.example	LEAD_ACTIVE_DAYS, ADMIN_ALERT_*
README_DEPLOY.md	Lead test messages + export docs

9. leads Table Schema

```
id, user_id, company_id, stage, lead_score
customer_name, location, budget, requested_speed, phone
contact_method, contact_value, urgency, buying_intent
coverage_check_needed, coverage_area, coverage_status
matched_product, alternative_product
```

```
last_discussed_product, last_sales_question, last_sales_reply
source_message, signals_json, created_at, updated_at
```

10. Environment Variables

```
COMPANY_ID=hons
LEAD_ACTIVE_DAYS=30
ADMIN_ALERT_ENABLED=false
ADMIN_TELEGRAM_CHAT_ID=
ADMIN_ALERT_MIN_SCORE=80
ADMIN_ALERT_STAGES=hot,qualified
```

11. Testing Checklist

#	Test	Expected
1	Internet jodnu cha	stage=new/interested, buying=true
2	Baluwatar ma 500 mbps internet jodnu cha	interested, coverage=true, NOT qualified
3	Phone + location + speed	stage=qualified
4	WhatsApp + location + speed	stage=qualified or hot
5	500 mbps (no catalog match)	sales mode, no full list
6	Support hours kati ho?	no lead upsert
7	Mero naam ke ho?	direct memory only
8	ADMIN_ALERT_ENABLED=true + hot lead	ADMIN_ALERT_PLACEHOLDER log

```
python -m unittest tests.test_lead_phase2 -v
python admin_export_leads.py
```

12. Deploy Checklist (Render)

1. Deploy ALL new + modified files together
2. Set COMPANY_ID, LEAD_ACTIVE_DAYS, ADMIN_ALERT_ENABLED
3. Restart service (creates leads table via init_db)
4. Send lead test messages via Telegram
5. Check logs: LEAD_SIGNALS_DETECTED, LEAD_UPSERTED, SALES_MODE_ACTIVE
6. Export: python admin_export_leads.py

13. Suggested Git Commits (10)

1. feat(leads): add Lead model to memory_db
2. feat(leads): add lead_manager persistence layer
3. feat(leads): add lead extraction, scoring, stage rules
4. feat(leads): product match and sales-mode catalog control
5. feat(leads): sales employee and coverage prompt blocks
6. feat(leads): buying intent and lead qualification intents
7. feat(leads): wire lead pipeline into reply engine
8. feat(leads): admin alert placeholder
9. feat(leads): admin lead export CLI
10. test(leads): phase 2 checklist and deploy docs

14. Remaining Technical Debt

- Admin alert is log-only — Phase 3: Telegram admin channel
- No CRM webhook export
- coverage_status always pending — no real coverage API
- create_all only — no Alembic migrations
- company_id not on user_memory table
- Blocking webhook handler unchanged

15. Recommended Phase 3

- Real admin Telegram notifications
- CRM webhook (Zoho/HubSpot)
- Background worker for webhook
- company_id on memory tables
- Coverage API integration for ISP